

RISC-V cheat sheet

Computer Architecture (KU Leuven), 2026–2027 — ISA & assembly (RV32I / RV64I + M), instruction encoding, calling conventions

Registers and ABI convention

Reg	ABI	Use by convention	Preserved?
x0	zero	hardwired to 0	—
x1	ra	return address for jumps	no
x2	sp	stack pointer	yes
x3	gp	global pointer	—
x4	tp	thread pointer	—
x5–x7	t0–t2	temporaries	no
x8	s0/fp	saved register 0 / frame pointer	yes
x9	s1	saved register 1	yes
x10–x11	a0–a1	return values / arguments 0–1	no
x12–x17	a2–a7	function arguments 2–7	no
x18–x27	s2–s11	saved registers 2–11	yes
x28–x31	t3–t6	temporaries	no

Notation

$R[rd]$ = register file access; $M_k[a]$ = k -byte memory access at address a ; $sxt(\cdot)/zxt(\cdot)$ = sign-/zero-extension to the machine word; $ea = R[rs1] + sxt(off)$; $\ll, \gg, \gg_a$ = shift left, logical / arithmetic shift right; $<_s, <_u$ = signed / unsigned compare; $\leftarrow$ = assignment. $(c) ? 1:0 = 1$ if c holds, else 0. $\dagger$ = RV64 only.

Computation instructions

Arithmetic		
add rd, rs1, rs2	$R[rd] \leftarrow R[rs1] + R[rs2]$	
sub rd, rs1, rs2	$R[rd] \leftarrow R[rs1] - R[rs2]$	
addi rd, rs1, imm	$R[rd] \leftarrow R[rs1] + sxt(imm)$	
addiw rd, rs1, imm†	$R[rd] \leftarrow sxt((R[rs1] + sxt(imm))[31:0])$	
Logical		
and rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \& R[rs2]$	
or rd, rs1, rs2	$R[rd] \leftarrow R[rs1] R[rs2]$	
xor rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \oplus R[rs2]$	
andi/ori/xori rd, rs1, imm	same, with $sxt(imm)$	
Shift		
sll rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \ll R[rs2]$	
srl rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \gg R[rs2]$	
sra rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \gg_a R[rs2]$	
slli/srli/srai rd, rs1, imm	same, with imm	
Comparison / PC-relative		
slt rd, rs1, rs2	$R[rd] \leftarrow (R[rs1] <_s R[rs2]) ? 1:0$	
sltu rd, rs1, rs2	$R[rd] \leftarrow (R[rs1] <_u R[rs2]) ? 1:0$	
slti/sltiu rd, rs1, imm	same, with $sxt(imm)$	
auipc rd, imm	$R[rd] \leftarrow PC + (imm \ll 12)$	
M extension (multiply/divide)		
mul rd, rs1, rs2	$R[rd] \leftarrow R[rs1] \times R[rs2]$ (low half)	
div rd, rs1, rs2	signed division	
rem rd, rs1, rs2	signed remainder	
divu, remu	unsigned division / remainder	
mulh, mulhu, mulhsu	upper half of the full product	

Data transfer instructions

lw rd, off(rs1)	$R[rd] \leftarrow sxt(M_4[ea])$
lwu rd, off(rs1)†	$R[rd] \leftarrow zxt(M_4[ea])$
sw rs2, off(rs1)	$M_4[ea] \leftarrow R[rs2]$
ld rd, off(rs1)†	$R[rd] \leftarrow M_8[ea]$
sd rs2, off(rs1)†	$M_8[ea] \leftarrow R[rs2]$
lh/lhu rd, off(rs1)	$R[rd] \leftarrow sxt/zxt(M_2[ea])$
sh rs2, off(rs1)	$M_2[ea] \leftarrow R[rs2]$
lb/lbu rd, off(rs1)	$R[rd] \leftarrow sxt/zxt(M_1[ea])$
sb rs2, off(rs1)	$M_1[ea] \leftarrow R[rs2]$
lui rd, imm	$R[rd] \leftarrow imm \ll 12$

Branch and jump instructions

beq rs1, rs2, label	$R[rs1] = R[rs2] \Rightarrow$ branch
bne rs1, rs2, label	$R[rs1] \neq R[rs2] \Rightarrow$ branch
blt rs1, rs2, label	$R[rs1] <_s R[rs2] \Rightarrow$ branch
bge rs1, rs2, label	$R[rs1] \geq_s R[rs2] \Rightarrow$ branch
bltu/bgeu rs1, rs2, label	unsigned variants
jal rd, label	$R[rd] \leftarrow PC + 4; \text{jump}$
jalr rd, off(rs1)	$R[rd] \leftarrow PC + 4; PC \leftarrow R[rs1] + sxt(off)$

System calls (ecall, RISC-V Linux)

Put the system call number in a7 and the arguments in a0–a5; the return value arrives in a0.

Call	a7	Arguments	Returns
read	63	a0=fd, a1=buf, a2=len	bytes read
write	64	a0=fd, a1=buf, a2=len	bytes written
exit	93	a0=status	—

File descriptors: 0 = stdin, 1 = stdout, 2 = stderr.

Pseudo-instructions

Pseudo	Expands to	Meaning
nop	addi zero, zero, 0	no operation
li rd, imm	addi rd, zero, imm	load immediate (12-bit)
li rd, big	lui + addi	load immediate (32-bit)
la rd, symbol	lui + addi	load address of symbol
mv rd, rs	addi rd, rs, 0	copy register
neg rd, rs	sub rd, zero, rs	negate
beqz rs, label	beq rs, zero, label	branch if zero
bnez rs, label	bne rs, zero, label	branch if $\neq$ zero
blez rs, label	bge zero, rs, label	branch if $\leq$ zero
j label	jal zero, label	jump
jr rs	jalr zero, 0(rs)	jump register
call symbol	jal ra, symbol	call function
ret	jalr zero, 0(ra)	return from call

Assembler directives

.global name	make label visible to other modules
.extern name	label is defined in another module*
.text	following lines go to the code section
.data	following lines go to the data section
.zero n	reserve n bytes, initialized to zero
.string "s"	store "s", terminated by a 0 byte
.word w, ...	store value(s) as 32-bit words
.half h, ...	store value(s) as 16-bit halfwords
.byte b, ...	store value(s) as bytes
.align n	align next item to a 2^n -byte boundary
.equ name, v	define constant name = v

Instruction encoding

	31–25	24–20	19–15	14–12	11–7	6–0
R	funct7	rs2	rs1	f3	rd	opc
I	imm[11:0]	rs1	f3	rd	opc	
S	imm[11:5]	rs2	rs1	f3	imm[4:0]	opc
B	imm[12]10:5]	rs2	rs1	f3	imm[4:1]11]	opc
U	imm[31:12]				rd	opc
J	imm[20]10:11]19:12]				rd	opc

Opcodes and function codes (course subset)

Instr	Fmt	opcode	funct3	funct7
add/sub	R	0110011	000	0000000/0100000
sll	R	0110011	001	0000000
slt/sltu	R	0110011	010/011	0000000
xor	R	0110011	100	0000000
srl/sra	R	0110011	101	0000000/0100000
or/and	R	0110011	110/111	0000000
mul/div/rem	R	0110011	000/100/110	0000001
addi	I	0010011	000	—
addiw†	I	0011011	000	—
slli/srli	I	0010011	001/101	0000000‡
srai	I	0010011	101	0100000‡
lb/lh/lw	I	0000011	000/001/010	—
lbu/lhu	I	0000011	100/101	—
ld†/ldu†	I	0000011	011/110	—
sb/sh/sw	S	0100011	000/001/010	—
sd†	S	0100011	011	—
beq/bne	B	1100011	000/001	—
blt/bge	B	1100011	100/101	—
bltu/bgeu	B	1100011	110/111	—
jalr	I	1100111	000	—
jal	J	1101111	—	—
lui/auipc	U	0110111/0010111	—	—
ecall	I	1110011	000	(imm = 0)

‡ for immediate shifts, imm[11:5] plays the role of funct7 and imm[4:0] is the shift amount.

Stack frame of a function call

fp (sp at entry) $\rightarrow$	(start of caller's frame)
	saved ra
	saved s0 (caller's fp)
	saved parameters
	...
	saved local variables
	...
sp $\rightarrow$	top of stack (grows down)